SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 9

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 9: The show page

Views, details and My List

Today ShowShelf gets a page for every show, with its cast and seasons, and a page for My List.

Instead of separate HTML files, the page now has three **views**: three `<section>` elements, of which only one shows at a time. Open the **index.html (whole page)** tab to see them: `#home-view`, `#show-view` and `#mylist-view`. Switching views is one more thing in the state.

Your code box starts with the finished app from Day 8, tidied into sections. The element lookups for the new parts of the page are already at the top (look for **Day 9**), so you can focus on what they do. Today's new code goes at the very bottom, under the line that says *new on Day 9*.

This is your finished app. On Day 11 it becomes your own website, online. Take your time, and read the code you add as well as writing it.

Step 1. Three views, one page ~20 min

Teaches: Views in one page: `state.view` plus `hidden` in `render()`. `event.stopPropagation()` stops bubbling.

TASK

1. In `state`, add `view: "home",`.
2. At the top of `render()`, add three lines that hide every view except the one in `state.view` (`homeView`, `showView` and `myListView`).
3. Fill in the TODO in `openShow`.
4. In `createCard`, before `return card;`, add a click listener to the card that calls `openShow(show.id)`.
5. In `createListButton`, give the listener an `event` parameter and call `event.stopPropagation();` first.

Click a card: the home view disappears. (The show view is empty for now.) Press **Run again** to go back.

COMMON MISTAKES

- `homeView.hidden = state.view === "home";`: the wrong way round, so the home view hides exactly when it should show.
- `card.addEventListener("click", openShow(show.id))`: this calls `openShow` straight away for every card.
- Forgetting `event` in `(event) => { ... }`, so `event.stopPropagation()` fails.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- State: everything on the page that can change ----------
const state = {
  view: "home",   // which view is showing: "home", "show" or "mylist"
  shows: [],      // the shows on the home page, from TVmaze (in our shape)
  genre: "",      // the chosen genre ("" means all genres)
// ... unchanged ...
  card.append(poster, body);
  card.append(createListButton(show));
  card.addEventListener("click", () => {
    openShow(show.id);
  });
  return card;
}
// ... unchanged ...
  button.setAttribute("aria-pressed", isInMyList(show.id));   // "pressed" look, and for screen
readers
  button.setAttribute("aria-label", "My List: " + show.name);
  button.addEventListener("click", (event) => {
    event.stopPropagation();   // a click on the button is not a click on the card
    toggleMyList(show);
  });
// ... unchanged ...

function render() {
  // Show the view that matches state.view, and hide the other two
  homeView.hidden = state.view !== "home";
  showView.hidden = state.view !== "show";
  myListView.hidden = state.view !== "mylist";

  let list = state.shows;
  if (state.genre !== "") {
// ... unchanged ...
```

```
// ========== The show view and My List (new on Day 9) ==========

// ---------- The show view ----------

// Open the show view for one show (a click on a card calls this)
function openShow(id) {
  state.view = "show";
  render();
  window.scrollTo(0, 0);   // start at the top of the page
}
```

Step 2. One show, with cast and seasons ~15 min

Teaches: `/shows/ID?embed[]=cast&embed[]=seasons` gets a show with its cast and seasons; a function can return an object.

TASK

1. Fill in the TODO in `getShowById`: return an object with three properties, `show`, `cast` and `seasons`.
2. Make `openShow` an `async function`, and at its end add `const data = await getShowById(id);` and `console.log(data.show.name);`.

Click a card and look at the Console.

COMMON MISTAKES

- `data.embedded.cast`: the property starts with an underscore, `_embedded`.
- Forgetting `async` on `openShow`, so `await` gives an error.
- `toShow(data._embedded)` instead of `toShow(data)`.

A CORRECT ANSWER

```
// ... unchanged ...

// Open the show view for one show (a click on a card calls this)
async function openShow(id) {
  state.view = "show";
  render();
  window.scrollTo(0, 0);   // start at the top of the page
  const data = await getShowById(id);
  console.log(data.show.name);
}

// Ask TVmaze for one show, with its cast and seasons in the same answer
// getShowById(431) -> { show: {...our shape...}, cast: [...], seasons: [...] }
async function getShowById(id) {
  const response = await fetch(`${API}/shows/${id}?embed[]=cast&embed[]=seasons`);
  if (!response.ok) {
    throw new Error("TVmaze answered " + response.status);
  }
  const data = await response.json();
  return {
    show: toShow(data),
    cast: data._embedded.cast,
    seasons: data._embedded.seasons,
  };
}
```

Step 3. Fill in the show ~20 min

Teaches: Fill a view from an object with `textContent`; never use `innerHTML` for text from outside.

TASK

1. Fill in the four TODOs in `renderShow`.
2. In `openShow`, replace `console.log(data.show.name);` with `renderShow(data.show);`.

Click a few cards: each opens with its own name, years, summary and genres. Press **Run again** to go back (the back button comes later).

COMMON MISTAKES

- `showSummary.innerHTML = show.summary`: works, but opens the door to code hidden in the text.
- Writing TODO 2 below the network lines, so `=` wipes the network out again.
- Appending the badges without `replaceChildren()` first (it's given), so they pile up from show to show.

A CORRECT ANSWER

```
// ... unchanged ...
  window.scrollTo(0, 0);   // start at the top of the page
  const data = await getShowById(id);
  renderShow(data.show);
}

// ... unchanged ...
  };
}

// Fill the show view with one show (in our shape)
function renderShow(show) {
  showPoster.src = show.image ?? "poster.svg";
  showPoster.alt = "Poster for " + show.name;
  showName.textContent = show.name;

  showMeta.textContent = yearsLabel(show.premiered, show.ended);
  if (show.network !== "") {
    showMeta.textContent += " · " + show.network;   // += adds to the end of the text
  }

  if (show.rating === null) {
    showRating.textContent = "No rating yet";
  } else {
    showRating.textContent = "★ " + show.rating + " · " + ratingLabel(show.rating);
  }

  // textContent, never innerHTML: the summary is text from the internet
  showSummary.textContent = show.summary;
  if (show.summary === "") {
    showSummary.textContent = "No summary yet.";
  }

  // One badge per genre
  showGenres.replaceChildren();
  for (const genre of show.genres) {
    const badge = document.createElement("span");
    badge.className = "badge";
    badge.textContent = genre;
    showGenres.append(badge);
    showGenres.append(" ");   // a space between the badges
  }
}
```

Step 4. The cast ~15 min

Teaches: Reach nested data step by step (`member.person.name`) and build a list with a loop.

TASK

1. Fill in the TODO in `renderCast`, so each cast member becomes an `<li>` like "Bryan Cranston as Walter White".
2. In `openShow`, under `renderShow(data.show);`, add `renderCast(data.cast);`.

COMMON MISTAKES

- `member.name`: the name is inside `person`.
- Creating the `li` outside the loop, so only the last person shows.

A CORRECT ANSWER

```
// ... unchanged ...
  const data = await getShowById(id);
  renderShow(data.show);
  renderCast(data.cast);
}

// ... unchanged ...
  }
}

// The cast as a list: "Jennifer Aniston as Rachel Green"
function renderCast(cast) {
  castList.replaceChildren();
  for (const member of cast) {
    const item = document.createElement("li");
    item.textContent = member.person.name + " as " + member.character.name;
    castList.append(item);
  }
  if (cast.length === 0) {
    const empty = document.createElement("li");
    empty.textContent = "No cast information yet.";
    castList.append(empty);
  }
}
```

Step 5. The seasons table ~15 min

Teaches: A small helper function (`createCell`) removes repetition.

TASK

1. Fill in the TODO in `renderSeasons`: add three cells to the row, for the season number, the number of episodes (`episodeOrder`) and the year.
2. In `openShow`, under `renderCast(data.cast);`, add `renderSeasons(data.seasons);`.

COMMON MISTAKES

- `createCell(season.premiereDate)`: the whole date instead of the year.
- Appending the row to `castList` or to the `table` instead of `seasonsBody`.

A CORRECT ANSWER

```
// ... unchanged ...
  renderShow(data.show);
  renderCast(data.cast);
  renderSeasons(data.seasons);
}

// ... unchanged ...
  }
}

// One table cell (<td>) with some text
function createCell(text) {
  const cell = document.createElement("td");
  cell.textContent = text;
  return cell;
}

// The seasons as table rows: season number, episodes, year
function renderSeasons(seasons) {
  seasonsBody.replaceChildren();
  for (const season of seasons) {
    const row = document.createElement("tr");
    row.append(
      createCell(season.number),
      createCell(season.episodeOrder),
      createCell(yearOf(season.premiereDate))
    );
    seasonsBody.append(row);
  }
}
```

Step 6. The back button ~10 min

Teaches: A back button changes `state.view` and calls `render()`.

TASK

At the bottom, add a click listener to `backButton` that sets `state.view` to `"home"` and calls `render()`.

Open a show, then go back: the same shows are still there, with the same genre chosen.

COMMON MISTAKES

- Calling `loadShows("")` in the back button: it works, but asks TVmaze again for nothing.

A CORRECT ANSWER

```
// ... unchanged ...
  }
}

// ---------- Switching views ----------

// The back button in the show view goes home
backButton.addEventListener("click", () => {
  state.view = "home";
  render();
});
```

Step 7. Loading and errors ~15 min

Teaches: Loading and error states for the show view: a message before `await`, `try / catch` around it.

TASK

In `openShow`, after `window.scrollTo(0, 0);`:

1. Hide the details, `showDetails.hidden = true;`, and show `showStatus` with the text "Loading show...". Also remove its `error` class.
2. Put the `await` line and the three `render...` lines inside `try { }`. At the end of `try`, hide the status and show the details again.
3. In `catch (error) { }`, give `showStatus` a friendly error message and the class `error`.

COMMON MISTAKES

- Showing the details before the `await`, so the old show flashes.
- Leaving `renderShow` outside `try`, so an error still breaks the page.

A CORRECT ANSWER

```
// ... unchanged ...
  render();
  window.scrollTo(0, 0);   // start at the top of the page

  // While loading: hide the old show and say what's happening
  showDetails.hidden = true;
  showStatus.textContent = "Loading show…";
  showStatus.classList.remove("error");
  showStatus.hidden = false;

  try {
    const data = await getShowById(id);
    renderShow(data.show);
    renderCast(data.cast);
    renderSeasons(data.seasons);
    showStatus.hidden = true;
    showDetails.hidden = false;
  } catch (error) {
    // No internet, or TVmaze answered with an error
    showStatus.textContent = "Sorry, this show could not be loaded (" + error.message + "). Check
your internet connection and try again.";
    showStatus.classList.add("error");
  }
}

// ... unchanged ...
```

Step 8. Show not found ~10 min

Teaches: Treat 404 as "not found" (`return null`), and use an early `return` for special cases.

TASK

1. In `getShowById`, before the `response.ok` check: if `response.status` is `404`, `return null;`.
2. In `openShow`, straight after the `await` line: if `data` is `null`, set `showStatus` to "Show not found." and `return`.

COMMON MISTAKES

- Putting the 404 check after `if (!response.ok)`: the function throws first and never gets there.
- Forgetting `return;` after the message, so `renderShow(null.show)` crashes.

A CORRECT ANSWER

```
// ... unchanged ...
  try {
    const data = await getShowById(id);
    if (data === null) {
      showStatus.textContent = "Show not found. Go back and pick another one.";
      return;
    }
    renderShow(data.show);
    renderCast(data.cast);
// ... unchanged ...

// Ask TVmaze for one show, with its cast and seasons in the same answer
// getShowById(431) -> { show: {...our shape...}, cast: [...], seasons: [...] }, or null if there's
no such show
async function getShowById(id) {
  const response = await fetch(`${API}/shows/${id}?embed[]=cast&embed[]=seasons`);
  if (response.status === 404) {
    return null;   // TVmaze has no show with this id
  }
  if (!response.ok) {
    throw new Error("TVmaze answered " + response.status);
// ... unchanged ...
```

Step 9. My List remembers more ~20 min

Teaches: Store small objects in localStorage; `some` checks whether any item matches.

TASK

1. In `state`, load the list with the new function: `myList: loadMyList(),`
2. In `saveMyList`, use the key `"myList"` again.
3. In `isInMyList`: `return state.myList.some((item) => item.id === id);`
4. In `toggleMyList`, remove with `state.myList.filter((item) => item.id !== show.id)`, and push an object with the five properties `id`, `name`, `image`, `genres` and `rating`, taken from `show`.

COMMON MISTAKES

- Changing `push` but not `filter` (or `isInMyList`), so taking a show out stops working.
- `includes(id)` on a list of objects: always `false`, because it compares whole objects with a number.
- Forgetting a comma between the properties of the object.

A CORRECT ANSWER

```
// ... unchanged ...
  shows: [],      // the shows on the home page, from TVmaze (in our shape)
  genre: "",      // the chosen genre ("" means all genres)
  myList: loadMyList(),   // small objects for the shows in My List (see loadMyList below)
};

// ... unchanged ...
// Is this show in My List? Gives true or false.
function isInMyList(id) {
  return state.myList.some((item) => item.id === id);
}

// ... unchanged ...
function toggleMyList(show) {
  if (isInMyList(show.id)) {
    state.myList = state.myList.filter((item) => item.id !== show.id);
  } else {
    // Save what a card needs, so My List can be drawn without asking TVmaze
    state.myList.push({
      id: show.id,
      name: show.name,
      image: show.image,
      genres: show.genres,
      rating: show.rating,
    });
  }
  saveMyList();
// ... unchanged ...
// Save My List in the browser (localStorage keeps text only, so we turn it into JSON text)
function saveMyList() {
  localStorage.setItem("myList", JSON.stringify(state.myList));
}

// ... unchanged ...
  render();
});

// ---------- My List, saved as small objects ----------
// Each show in My List is saved with what a card needs, so the My List view
// can draw it without asking TVmaze:
//   { id: 431, name: "Friends", image: null, genres: ["Comedy", "Romance"], rating: 8.8 }

// Read My List from localStorage (the key is "myList")
```

```
function loadMyList() {
  const saved = JSON.parse(localStorage.getItem("myList")) ?? [];
  // On Day 7 this key held plain ids, like [2, 5]. Keep only the objects.
  return saved.filter((item) => typeof item === "object");
}
```

Step 10. The My List view ~20 min

Teaches: Reuse `createCard` for a second view; `preventDefault()` on links you handle yourself.

TASK

1. **TODO 1** in `renderMyList`: a `for...of` loop over `state.myList` that appends `createCard(show)` to `myListGrid`.
2. At the end of `render()`, call `renderMyList();`, so My List is drawn every time the state changes.
3. **TODO 2**: click listeners for `homeLink` (to the `"home"` view) and `myListLink` (to the `"mylist"` view), like the logo's.

Add a few shows, open **My List** in the header, and take one out again.

COMMON MISTAKES

- Forgetting `renderMyList()` in `render()`: the view only updates on a reload.
- Leaving out `event.preventDefault()`: in the Lab a notice appears; on the real website the page would reload.
- Looping over `state.shows` instead of `state.myList`.

A CORRECT ANSWER

```
// ... unchanged ...
  renderGenres();
  listCount.textContent = state.myList.length;
  renderMyList();
}

// ... unchanged ...
  return saved.filter((item) => typeof item === "object");
}

// ---------- The My List view ----------

// Draw the saved shows, with the same createCard as the home page
function renderMyList() {
  myListGrid.replaceChildren();
  for (const show of state.myList) {
    myListGrid.append(createCard(show));
  }
  myListEmpty.hidden = state.myList.length > 0;
}

// The logo goes home
logoLink.addEventListener("click", (event) => {
  event.preventDefault();   // don't follow the link: we switch views ourselves
  state.view = "home";
  render();
});

// The Home link goes home too
homeLink.addEventListener("click", (event) => {
  event.preventDefault();
  state.view = "home";
  render();
});

// The My List link opens the My List view
myListLink.addEventListener("click", (event) => {
  event.preventDefault();
  state.view = "mylist";
```

```
  render();
});
```

Step 11. My List from the show view ~15 min

Teaches: `state.currentShow` remembers the open show; one `render()` keeps every view in agreement.

TASK

1. In `state`, add `currentShow: null,`.
2. In `openShow`, right before `renderShow(data.show);`, remember the show: `state.currentShow = data.show;`. Right after `renderSeasons(...)`, call `renderShowListButton();`.
3. At the end of `render()`, call `renderShowListButton();`.
4. At the bottom, add a click listener to `showListButton` that calls `toggleMyList(state.currentShow);`.

Open a show, add it, then look at its card on the home page and in My List: all three agree.

COMMON MISTAKES

- Setting the button's text inside the click listener instead of in `render()`: it goes wrong when the show is added from its card.
- Adding the listener inside `openShow`: every opened show adds one more listener, and one click toggles several times.

NOTE

This is the last core step of the app students take home on Day 11. If time allows, ask them to try it online (the preview asks the real TVmaze when there's internet): real posters, and some shows with "No rating yet".

A CORRECT ANSWER

```
// ... unchanged ...
  genre: "",      // the chosen genre ("" means all genres)
  myList: loadMyList(),   // small objects for the shows in My List (see loadMyList below)
  currentShow: null,      // the show open in the show view (null when none is)
};

// ... unchanged ...
  listCount.textContent = state.myList.length;
  renderMyList();
  renderShowListButton();
}

// ... unchanged ...
      return;
    }
    state.currentShow = data.show;
    renderShow(data.show);
    renderCast(data.cast);
    renderSeasons(data.seasons);
    renderShowListButton();
    showStatus.hidden = true;
    showDetails.hidden = false;
// ... unchanged ...
  render();
});

// The My List button in the show view. Its text follows the state.
function renderShowListButton() {
  if (!state.currentShow) {
    return;   // no show is open yet
  }
  if (isInMyList(state.currentShow.id)) {
    showListButton.textContent = "✓ In My List";
```

```
  } else {
    showListButton.textContent = "+ Add to My List";
  }
  showListButton.setAttribute("aria-pressed", isInMyList(state.currentShow.id));
}

// Add the open show to My List, or take it out
showListButton.addEventListener("click", () => {
  toggleMyList(state.currentShow);
});
```

Extra 1. A link for every show ~20 min

Teaches: `location.hash` and the `"hashchange"` event give each view its own address.

TASK

1. **TODO**: call `showFromAddress` on every `"hashchange"` of `window`.
2. In `createCard`'s click listener, replace `openShow(show.id);` with `location.hash = "show/" + show.id;`.
3. In the back button's listener, replace its two lines with `location.hash = "";`.
4. In the logo, Home and My List listeners, add `location.hash = "";` after `event.preventDefault();`, so the address forgets the show.

NOTE

Inside the Lab, the preview's own address changes (you can't see it). On the real website from Day 11, the address bar shows `#show/431` and the browser's back button works.

A CORRECT ANSWER

```
// ... unchanged ...
  card.append(createListButton(show));
  card.addEventListener("click", () => {
    location.hash = "show/" + show.id;   // the hashchange listener opens the show
  });
  return card;
// ... unchanged ...
// The back button in the show view goes home
backButton.addEventListener("click", () => {
  location.hash = "";   // the hashchange listener goes home
});

// ... unchanged ...
logoLink.addEventListener("click", (event) => {
  event.preventDefault();   // don't follow the link: we switch views ourselves
  location.hash = "";
  state.view = "home";
  render();
// ... unchanged ...
homeLink.addEventListener("click", (event) => {
  event.preventDefault();
  location.hash = "";
  state.view = "home";
  render();
// ... unchanged ...
myListLink.addEventListener("click", (event) => {
  event.preventDefault();
  location.hash = "";
  state.view = "mylist";
  render();
// ... unchanged ...
  toggleMyList(state.currentShow);
});

// ---------- Extra: an address for every show, like index.html#show/431 ----------

// Show the view that the address asks for
function showFromAddress() {
  if (location.hash.startsWith("#show/")) {
    const id = Number(location.hash.slice(6));   // "#show/431" -> 431
    openShow(id);
  } else if (state.view === "show") {
```

```
    state.view = "home";                          // the address has no show any more
    render();
  }
}

window.addEventListener("hashchange", showFromAddress);

// Someone opened a link to a show, like index.html#show/431: open it straight away
if (location.hash.startsWith("#show/")) {
  showFromAddress();
}
```

Extra 2. Cast photos ~15 min

Teaches: `append` mixes elements and text; check for `null` before using an optional object.

TASK

In `renderCast`'s loop, when `member.person.image` is not `null`, create an `<img>` with `src` set to `member.person.image.medium`, an empty `alt`, and `width` 60, and append it to the `li` **before** the text. Then append the text with `append` instead of `textContent`.

A CORRECT ANSWER

```
// ... unchanged ...
  for (const member of cast) {
    const item = document.createElement("li");
    if (member.person.image !== null) {
      const photo = document.createElement("img");
      photo.src = member.person.image.medium;
      photo.alt = "";
      photo.width = 60;
      item.append(photo);
    }
    item.append(member.person.name + " as " + member.character.name);
    castList.append(item);
  }
// ... unchanged ...
```

Extra 3. The average rating of My List ~15 min

Teaches: Average: filter out `null`, add up in a loop, divide, `toFixed(1)`.

TASK

1. At the top: `const myListAverage = document.querySelector("#mylist-average");`
2. At the end of `renderMyList`: keep the shows whose rating isn't `null`. If there are none, empty the line. Otherwise add up their ratings and show "Average rating: " with the average, to one decimal.

A CORRECT ANSWER

```
// ... unchanged ...
const myListGrid = document.querySelector("#mylist-grid");
const myListEmpty = document.querySelector("#mylist-empty");
const myListAverage = document.querySelector("#mylist-average");

// ---------- State: everything on the page that can change ----------
// ... unchanged ...
  }
  myListEmpty.hidden = state.myList.length > 0;

  // The average rating of the shows that have one
  const rated = state.myList.filter((show) => show.rating !== null);
  if (rated.length === 0) {
    myListAverage.textContent = "";
  } else {
    let total = 0;
    for (const show of rated) {
      total = total + show.rating;
    }
    myListAverage.textContent = "Average rating: " + (total / rated.length).toFixed(1);
  }
}

// ... unchanged ...
```